

StockIQ

WMS · ERP LITE

Documentação Técnica de Planejamento — Sprint 1

Projeto Integrador VI · Computação em Nuvem II · Mineração de Dados

Gabriel da Silveira Pessoni

Lívia Portela Ferreira

FATEC — Faculdade de Tecnologia de Franca – Desenvolvimento de Software
Multiplataforma

Sumário

1. Introdução
2. Escopo do Projeto e Requisitos
3. Modelagem Inicial — Casos de Uso e Arquitetura
4. Estrutura Inicial do Back-end
5. Protótipo Inicial do Front-end
6. Banco de Dados — Modelo Conceitual e Lógico
7. Computação em Nuvem II — Arquitetura AWS
8. Mineração de Dados
9. Status Consolidado da Sprint 1 e Próximos Passos
10. Considerações Finais

SOBRE ESTE DOCUMENTO

Este relatório consolida as entregas mínimas definidas para a 1ª Sprint (04/09/2026) do Projeto Integrador, integrando os componentes curriculares de **Computação em Nuvem II** e **Mineração de Dados**. O conteúdo técnico (rotas, modelos de dados, telas e stack) foi extraído diretamente do estado atual dos repositórios [P.I-6-Semestre-Backend](#) e [P.I-6-Semestre-Frontend](#), refletindo o que já está implementado — e não apenas um planejamento teórico.

1. Introdução

O **StockIQ** é um sistema de gestão de estoque e armazém (WMS — *Warehouse Management System*) desenvolvido como Projeto Integrador do 6º semestre. O sistema tem como objetivo digitalizar e centralizar o controle de produtos, lotes, movimentações de entrada/saída, endereçamento físico em armazém, fornecedores, clientes e inventário, oferecendo ainda relatórios gerenciais e um módulo de inteligência analítica para apoiar decisões de reposição de estoque.

O projeto é dividido em dois repositórios independentes que se comunicam via API REST:

- **P.I-6-Semestre-Backend** — API REST em Node.js/Express, TypeScript e PostgreSQL (via Prisma ORM);
- **P.I-6-Semestre-Frontend** — aplicação web em Next.js/React, responsável pela interface de operação do sistema.

Esta primeira sprint tem caráter estrutural: o foco não é entregar funcionalidades finais polidas, e sim consolidar a base do projeto — escopo, modelagem, esqueleto do back-end, protótipo navegável do front-end, modelagem do banco de dados e as definições de infraestrutura em nuvem e de mineração de dados que sustentarão as próximas sprints.

Item	Descrição
Nome do sistema	StockIQ — WMS / ERP Lite
Domínio	Gestão de estoque, armazenagem, movimentações e inventário
Tipo de solução	Aplicação web (API REST + SPA/SSR), multiusuário, com controle de perfis de acesso
Disciplinas integradas	Projeto Integrador VI, Computação em Nuvem II, Mineração de Dados
Infraestrutura alvo	AWS (VPS/EC2, Load Balancer, mensageria e armazenamento gerenciado)

2. Escopo do Projeto e Requisitos

2.1 Escopo

O StockIQ cobre o ciclo completo de operação de um armazém de médio porte: cadastro de produtos e suas categorias, marcas, fornecedores e clientes; endereçamento físico do estoque (corredor, rua, prateleira, nível, posição); registro de movimentações (entrada, saída, transferência, perda, ajuste e inventário); controle de lotes com data de validade; contagens periódicas de inventário com apuração de divergências; alertas automáticos de estoque baixo e de vencimento; trilha de auditoria de operações críticas; dashboard e relatórios gerenciais (incluindo curva ABC); e um módulo de IA analítica voltado à previsão de demanda e sugestão de reposição. Ficam fora do escopo desta primeira versão: emissão fiscal (NF-e), integração com marketplaces e pagamentos — itens que podem compor evoluções futuras.

2.2 Requisitos Funcionais

Código	Descrição	Status
RF01	Permitir cadastro e autenticação de usuários (e-mail/senha) com emissão de token JWT.	IMPLEMENTADO
RF02	Controlar acesso por perfil de usuário (Administrador, Supervisor, Operador, Visualizador), restringindo criação/edição/exclusão conforme o papel.	IMPLEMENTADO
RF03	Cadastrar, editar, consultar e remover produtos (SKU, código interno, código de barras, dimensões, peso, preços, estoque mínimo/máximo).	IMPLEMENTADO
RF04	Consultar produto por leitura de código de barras (scanner).	IMPLEMENTADO
RF05	Cadastrar categorias e marcas de produtos.	IMPLEMENTADO
RF06	Cadastrar fornecedores e clientes (dados cadastrais, CNPJ, contato).	IMPLEMENTADO
RF07	Cadastrar endereços de armazenagem com status (livre, ocupado, bloqueado, reservado) e capacidade.	IMPLEMENTADO

RF08	Registrar movimentações de estoque (entrada, saída, transferência, perda, ajuste, inventário) vinculadas a produto, usuário, fornecedor/cliente e endereços de origem/destino.	IMPLEMENTADO
RF09	Controlar lotes com datas de fabricação/validade e status automático (válido, a vencer, vencido, quarentena).	IMPLEMENTADO
RF10	Planejar e executar contagens de inventário (completa, parcial, cíclica), divergências.	IMPLEMENTADO registrando
RF11	Emitir alertas automáticos de estoque abaixo do mínimo e de lotes próximos ao vencimento.	IMPLEMENTADO
RF12	Manter log de auditoria (ação, entidade, valor antigo/novo, usuário, IP, data/hora).	IMPLEMENTADO
RF13	Exibir dashboard com resumo geral, movimentações recentes e produtos mais movimentados.	IMPLEMENTADO
RF14	Gerar relatórios de movimentações, posição de estoque, validade de lotes, ocupação de armazém, curva ABC, inventário e compras por fornecedor.	IMPLEMENTADO
RF15	Disponibilizar módulo de IA Analítica com previsão de demanda, classificação ABC/XYZ e sugestões de reposição.	PROTÓTIPO (DADOS MOCK)
RF16	Disponibilizar documentação interativa da API (Swagger/OpenAPI).	IMPLEMENTADO

2.3 Requisitos Não Funcionais

Código	Descrição
RNF01	Segurança — senhas com hash (bcrypt), autenticação JWT, cabeçalhos de segurança HTTP (Helmet), CORS controlado e limitação de taxa de requisições (rate limiting).
RNF02	Validação — toda entrada da API é validada em camada própria (Zod) antes de chegar à regra de negócio.
RNF03	Disponibilidade — infraestrutura em nuvem com balanceamento de carga entre múltiplas instâncias.
RNF04	Escalabilidade — arquitetura apta a escalar horizontalmente o back-end conforme aumento de carga.
RNF05	Desempenho — listagens com paginação e filtros para evitar sobrecarga em grandes volumes.
RNF06	Responsividade — front-end utilizável em desktop e dispositivos móveis (leitura de código de barras em coletores/celulares).
RNF07	Manutenibilidade — código em camadas (rotas, controllers, services, schemas, middlewares), 100% tipado em TypeScript.
RNF08	Testabilidade — cobertura de testes automatizados (Jest + Supertest) nos principais fluxos da API.

RNF09 **Rastreabilidade** — toda alteração crítica é auditável via log de auditoria.

RNF10 **Observabilidade** — logs de aplicação (Morgan) e métricas de infraestrutura monitoradas (CloudWatch, na nuvem).

STATUS DESTA ENTREGA

A definição de escopo e requisitos está **concluída** para a Sprint 1. Novos requisitos poderão ser incorporados nas próximas sprints conforme validação com o orientador e evolução do módulo de IA analítica.

3. Modelagem Inicial

3.1 Diagrama de Casos de Uso

O acesso ao sistema é hierárquico: cada perfil herda as permissões do perfil abaixo dele, acrescentando novas capacidades. Essa hierarquia reflete diretamente as regras de autorização implementadas nas rotas da API.

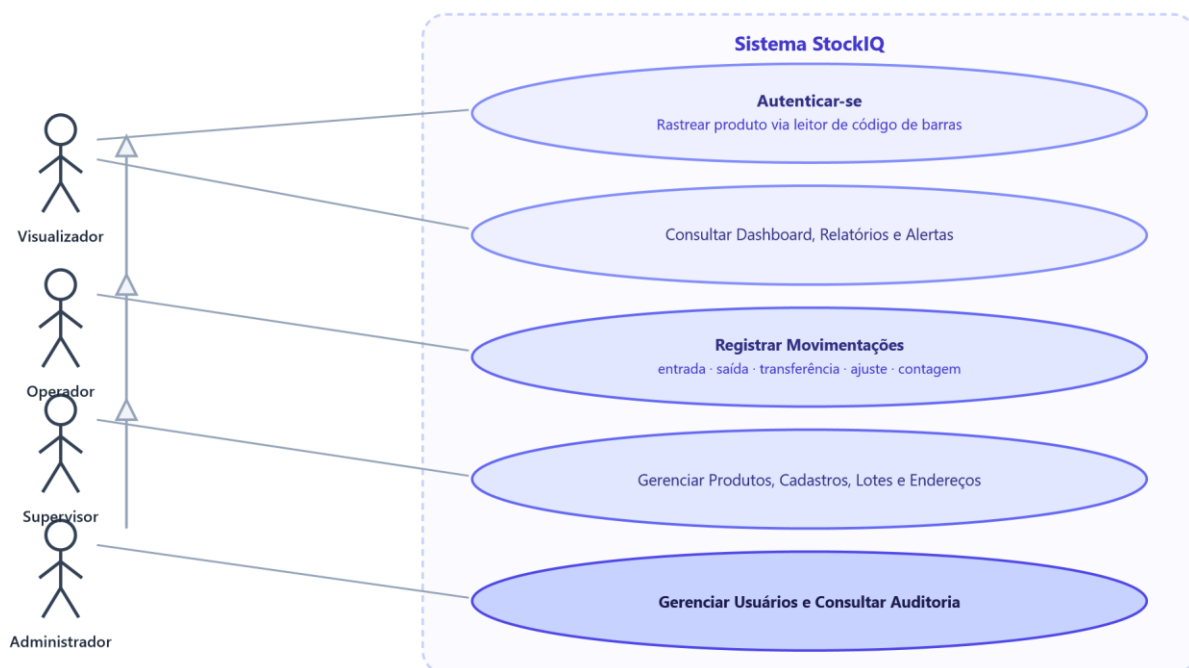


Figura 1 — Diagrama de casos de uso simplificado. Cada perfil herda as permissões do perfil imediatamente abaixo (generalização de atores).

3.2 Arquitetura da Solução (visão lógica)

A solução segue uma arquitetura cliente-servidor desacoplada: o front-end consome exclusivamente a API REST do back-end, que por sua vez conversa com o banco de dados relacional através do Prisma ORM.



Hospedado em infraestrutura AWS (ver Capítulo 7) — front-end e back-end publicados como serviços independentes

Figura 2 — Visão lógica da arquitetura da aplicação.

STATUS DESTA ENTREGA

Diagramas de caso de uso e arquitetura lógica **concluídos**. Diagrama de classes e diagramas de sequência dos fluxos críticos (registro de movimentação, alerta de estoque) ficam para a próxima sprint, à medida que o módulo de IA analítica for integrado.

4. Estrutura Inicial do Back-end

O back-end foi estruturado como uma API REST em camadas (rotas → controllers → services → validação → banco de dados), o que facilita manutenção, testes e a futura divisão de responsabilidades entre a equipe.

4.1 Stack Tecnológica

Camada	Tecnologia	Finalidade
Linguagem	TypeScript	Tipagem estática, redução de erros em tempo de execução
Framework HTTP	Express.js	Roteamento e middlewares da API REST
ORM	Prisma 6	Modelagem, migrations e acesso ao PostgreSQL
Autenticação	jsonwebtoken + bcryptjs	Login com token JWT e hash de senha
Validação	Zod	Validação de payloads de entrada por schema
Segurança	Helmet, CORS, express-rate-limit	Cabeçalhos seguros, controle de origem e limite de requisições
Documentação	swagger-jsdoc + swagger-uiexpress	Documentação interativa da API (/docs)
Testes	Jest + Supertest	Testes automatizados de integração dos endpoints
Logs	Morgan	Log de requisições HTTP em desenvolvimento

4.2 Módulos e Endpoints Implementados

A própria API expõe, em sua rota raiz, um painel de documentação que contabiliza **15 módulos e 67 endpoints REST**, com autenticação via JWT e 3 níveis efetivos de controle de acesso (público, autenticado padrão, supervisor e administrador). Os módulos implementados são:

Módulo	Base da rota	Resumo
Autenticação	/api/auth	Registro, login, perfil do usuário autenticado, troca de senha
Usuários	/api/users	CRUD de usuários (restrito a administrador)
Produtos	/api/products	CRUD de produtos, listagem com filtros/paginação, busca por código de barras
Categorias	/api/categories	CRUD de categorias
Marcas	/api/brands	CRUD de marcas
Fornecedores	/api/suppliers	CRUD de fornecedores
Clientes	/api/customers	CRUD de clientes, com busca e paginação
Armazéns	/api/warehouse	CRUD de endereços físicos de armazenagem
Movimentações	/api/movements	Registro e consulta de movimentações de estoque

Lotes	<code>/api/lots</code>	CRUD de lotes com controle de validade
Inventário	<code>/api/inventory</code>	Posição de estoque, estoque baixo, lotes a vencer, histórico por produto
Dashboard	<code>/api/dashboard</code>	Resumo geral, movimentações recentes, produtos mais movimentados
Alertas	<code>/api/alerts</code>	Alertas de estoque e validade, marcação de leitura
Auditoria	<code>/api/audit</code>	Consulta de logs de auditoria (restrito a administrador)
Relatórios	<code>/api/reports</code>	Movimentações, estoque, lotes, ocupação, curva ABC, inventário, fornecedores

4.3 Segurança e Qualidade

- Autorização por papel (`admin` / `supervisor` / autenticado) aplicada por middleware em cada rota sensível;
 - Validação de entrada centralizada via schemas Zod, um por módulo (`src/schemas`);
 - Middleware de auditoria (`audit.middleware.ts`) registrando alterações relevantes;
 - Suíte de testes automatizados cobrindo os fluxos de autenticação, produtos, categorias, movimentações, usuários e armazéns (`__tests__` /);
 - Tratamento centralizado de erros (`error.middleware.ts`) e limitação global de requisições (`ratelimit.middleware.ts`).

STATUS DESTA ENTREGA

Estrutura do back-end **concluída** para a Sprint 1: API funcional, autenticada, validada, documentada e testada. Próximos passos: implementar a fila de mensageria assíncrona (Capítulo 7) e o pipeline de exportação de dados para o módulo de mineração (Capítulo 8).

5. Protótipo Inicial do Front-end

O front-end foi iniciado como uma aplicação Next.js (App Router) com React 19 e Tailwind CSS 4, priorizando um protótipo navegável de todas as telas do sistema antes da integração completa com a API real.

5.1 Stack Tecnológica

Camada	Tecnologia
Framework	Next.js 16 (App Router)
Biblioteca de UI	React 19
Estilização	Tailwind CSS 4
Ícones	lucide-react
Gráficos	Recharts

5.2 Telas Implementadas (protótipo estático)

As telas abaixo já estão estruturadas com layout, componentes reutilizáveis (`Card` , `Badge` , `Button` , `Breadcrumb` , sidebar e topbar responsivos) e dados de exemplo (`mocks/`), enquanto a integração com a API real está em andamento:

- Login (autenticação)
- Entradas e Saídas de estoque
- Dashboard geral (indicadores e gráficos)
- Movimentações (histórico geral)
- Produtos (listagem, cadastro, detalhe)
- Lotes (controle de validade)
- Categorias e Marcas
- Inventário (contagens)
- Fornecedores e Clientes
- Alertas e Auditoria
- Endereços de armazém
- Relatórios, Scanner, Usuários, Configurações, IA / IA Analítica

DESTAQUE — MÓDULO IA ANALÍTICA

A tela `ia-analitica` já prototipa, com dados de exemplo, os principais entregáveis do Capítulo 8 (Mineração de Dados): gráfico de previsão de demanda (real vs. previsto), matriz de classificação ABC/XYZ e cartões de insights/sugestões automáticas de compra. Isso antecipa visualmente o que os modelos de mineração precisarão alimentar com dados reais nas próximas sprints.

STATUS DESTA ENTREGA

Protótipo inicial do front-end **concluído** — todas as telas centrais navegáveis com dados estáticos.

Próximo passo: substituir os mocks por chamadas reais à API REST do back-end (autenticação, CRUDs e relatórios).

6. Banco de Dados — Modelo Conceitual e Lógico

6.1 Modelo Conceitual (simplificado)

O modelo abaixo apresenta as entidades centrais do domínio de estoque e seus relacionamentos principais. Entidades de apoio (notificações e logs de auditoria) foram omitidas do diagrama por clareza, mas estão detalhadas no modelo lógico (item 6.2).

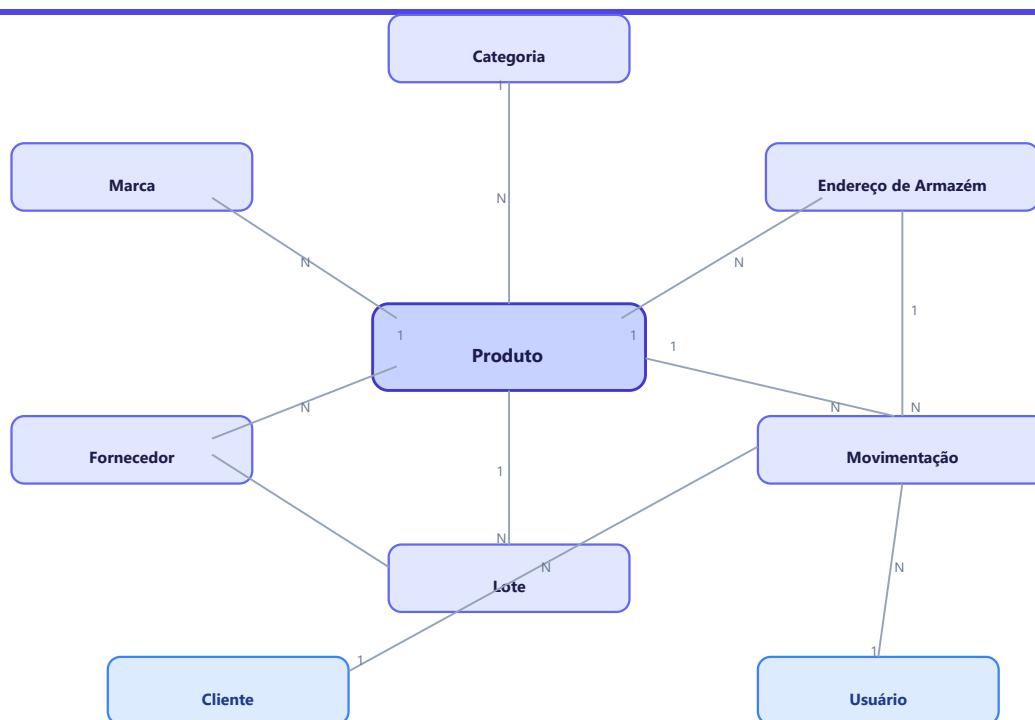


Figura 3 — Modelo conceitual simplificado (entidades centrais e cardinalidades).

6.2 Modelo Lógico

O modelo lógico já está implementado via **Prisma Schema** e versionado por migrations, totalizando 12 entidades e 11 enumerações de domínio. As tabelas centrais são detalhadas a seguir; as demais são resumidas na Tabela 6.5. **products**

Campo	Tipo	Observações
id	String (cuid)	PK
name, internalCode, sku, barcode	String	sku, barcode e internalCode únicos

unit, weight, width, height, depth	String / Float	Dados logísticos do item
purchasePrice, salePrice	Float	Precificação
minStock, maxStock, currentStock	Int	Base para alertas de ruptura/excesso
status	Enum ProductStatus	active · inactive · discontinued
categoryId, brandId, supplierId	String	FK → Category, Brand, Supplier

movements

Campo	Tipo	Observações
id	String (cuid)	PK
type	Enum MovementType	entry · exit · transfer · loss · adjustment · inventory
quantity, unitCost, totalValue	Int / Float	Base para relatórios de valor e curva ABC
exitReason	Enum ExitReason	sale · transfer · loss · break · internal
productId, userId	String	FK → Product, User
supplierId, customerId	String?	FK opcionais conforme o tipo de movimentação
fromAddressId, toAddressId	String?	FK → WarehouseAddress (origem/destino)
createdAt	DateTime	Base temporal para séries históricas (mineração de dados)

lots · warehouse_addresses

Tabela	Principais campos	Observações
lots	lotNumber, quantity, manufacturingDate, expirationDate, status, productId, supplierId	status calculado: valid · expiring · expired · quarantine
warehouse_addresses	code, aisle, street, shelf, level, position, status, capacity, occupied, productId	status: free · occupied · blocked · reserved

6.5 Demais entidades

Entidade	Principais campos	Relacionamentos
----------	-------------------	-----------------

users	name, email, password (hash), role, department, status, lastLogin	1:N com Movement, InventoryCount, Notification, AuditLog
categories	name, slug, color	1:N com Product
brands	name, slug, logoUrl	1:N com Product
suppliers	name, tradeName, cnpj, email, phone, category, status	1:N com Product, Movement, Lot
customers	name, tradeName, cnpj, email, city, state, status	referenciado por Movement (customerId/customerName)
inventory_counts	type, status, startDate, endDate, totalItems, countedItems, divergences	N:1 com User (responsável)
notifications	alertKey, readAt	N:1 com User; único por (alertKey, userId)
audit_logs	action, entity, entityId, oldValue (JSON), newValue ip	N:1 com User (JSON),

6.6 Enumerações de domínio

Enum	Valores
UserRole	admin · supervisor · operator · viewer
MovementType	entry · exit · transfer · loss · adjustment · inventory
LotStatus	valid · expiring · expired · quarantine
PositionStatus	free · occupied · blocked · reserved
InventoryCountType / Status	full · partial · cyclic / planned · in_progress · review · completed

STATUS DESTA ENTREGA

Modelo conceitual e lógico **concluídos** e implantados via Prisma Migrate. Próximo passo: modelagem de índices adicionais para consultas analíticas (relatórios e mineração de dados) e avaliação de particionamento da tabela [movements](#) conforme o volume de dados crescer.

7. Computação em Nuvem II — Arquitetura AWS

A infraestrutura do StockIQ será hospedada na **AWS**, utilizando instâncias **EC2** como VPS tanto para a aplicação quanto para o banco de dados, com um **Load Balancer** distribuindo o tráfego entre as instâncias da API. Esta seção justifica cada serviço escolhido e propõe usos concretos de mensageria assíncrona para o projeto.

7.1 Diagrama de Arquitetura

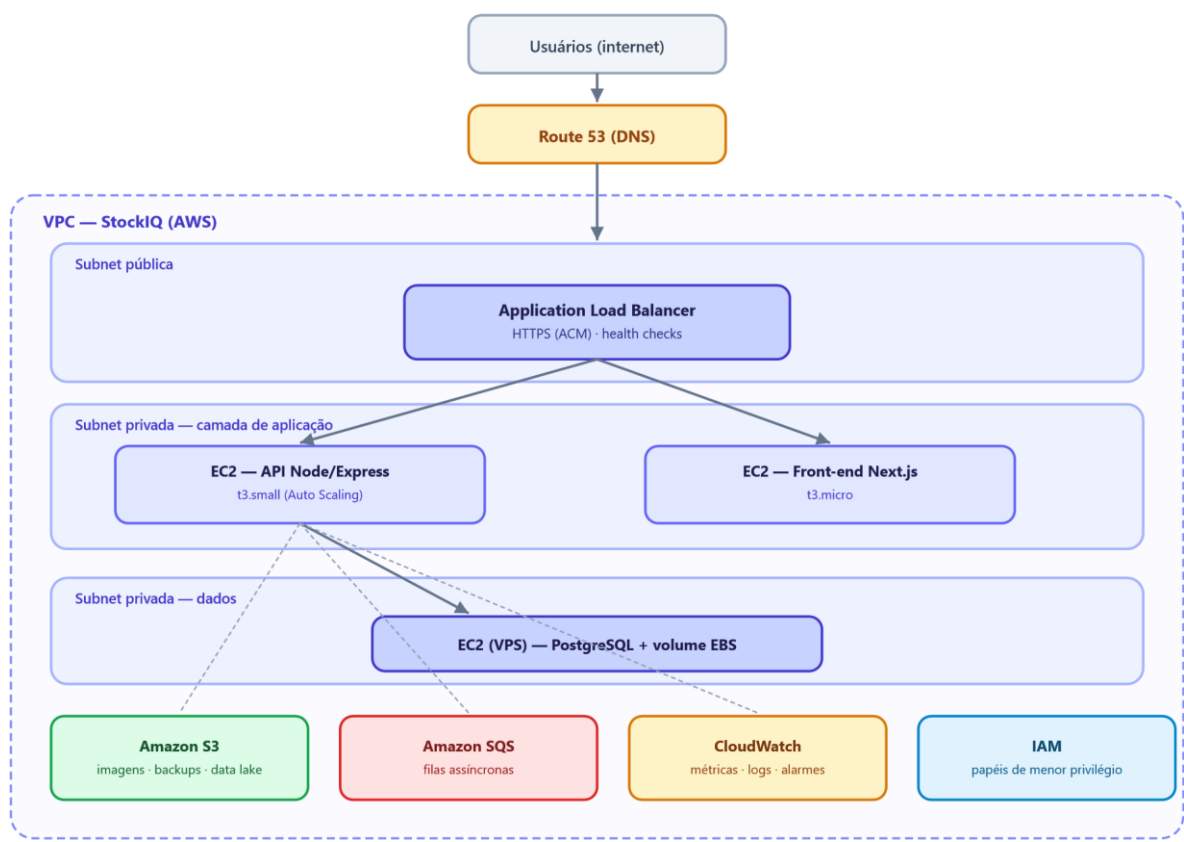


Figura 4 — Arquitetura de infraestrutura proposta na AWS. Linhas tracejadas indicam integrações assíncronas/gerenciadas a partir da API.

7.2 Justificativa dos Serviços

Serviço AWS	Justificativa de uso no StockIQ
-------------	---------------------------------

EC2 (VPS)	Hospeda a API Node.js e, em instância separada, o banco PostgreSQL — modelo pedido pela equipe (VPS próprio em vez de serviços totalmente gerenciados), com controle total do ambiente e menor custo em nível educacional/free tier.
Application Load Balancer	Distribui requisições entre as instâncias EC2 da API, permite alta disponibilidade (health checks removem instâncias com falha) e possibilita escalar horizontalmente sem downtime nas próximas sprints.
Auto Scaling Group (próximos passos)	Adiciona/remove instâncias de API automaticamente conforme CPU/memória, mantendo custo baixo em horários de menor uso.
VPC + Subnets públicas/privadas	Isola o banco de dados e as instâncias de aplicação da internet pública; apenas o Load Balancer fica exposto, reduzindo superfície de ataque.
Security Groups	Regras de firewall por camada: ALB aceita 443 da internet; API aceita tráfego apenas do ALB; banco aceita tráfego apenas das instâncias de API.
Amazon S3	Armazenamento de imagens de produtos/avatars, backups agendados do PostgreSQL (<code>pg_dump</code>) e, futuramente, o "data lake" bruto para o módulo de mineração de dados (Capítulo 8).
Amazon SQS	Ver detalhamento no item 7.3 — desacopla processamento assíncrono do fluxo síncrono da API.
Route 53	Gerenciamento de domínio e DNS do sistema, apontando para o Load Balancer.
AWS Certificate Manager	Certificado TLS gratuito para HTTPS no Load Balancer.
CloudWatch	Monitoramento de CPU/memória das instâncias, logs centralizados da API e alarmes (ex.: uso de disco do banco, filas SQS acumulando mensagens).
IAM	Papéis com permissões mínimas necessárias para cada instância acessar S3/SQS, seguindo o princípio do menor privilégio.

7.3 Mensageria (Amazon SQS) — usos propostos

A equipe cogitou usar mensageria "para algo" sem uma aplicação definida; três usos concretos e justificados para este sistema foram identificados:

1. Fila de alertas e notificações

Quando uma movimentação deixa o estoque abaixo do mínimo ou um lote se aproxima do vencimento, a API publica uma mensagem na fila em vez de processar o envio de e-mail/notificação de forma síncrona. Um worker consome a fila e envia a notificação, evitando que a requisição do usuário fique lenta esperando esse envio.

2. Geração assíncrona de relatórios pesados

Relatórios como a curva ABC ou exportações em PDF/Excel podem demorar com grande volume de dados. A API enfileira o pedido, um worker processa em segundo plano e o resultado fica disponível para download — mantendo a API responsiva mesmo sob carga.

3. Pipeline de eventos para mineração de dados

Cada movimentação de estoque publica um evento na fila, consumido por um processo que grava o registro em lote no "data lake" (S3). Isso desacopla a origem operacional dos dados do processo de preparação usado pela mineração de dados (Capítulo 8), sem sobrecarregar o banco transacional com leituras analíticas.

STATUS DESTA ENTREGA

Arquitetura em nuvem **definida e justificada**. Próximos passos: provisionamento efetivo dos recursos (idealmente via Infraestrutura como Código — Terraform ou CloudFormation), deploy das aplicações nas instâncias EC2 e configuração do Load Balancer e da primeira fila SQS.

8. Mineração de Dados

8.1 Definição da Base de Dados

A base de dados para mineração será formada pelos próprios **dados operacionais gerados pelo uso do StockIQ** — não haverá uma base externa. As tabelas `movements`, `products`, `lots`, `suppliers` e `warehouse_addresses` (detalhadas no Capítulo 6) concentram os atributos com maior potencial analítico: tipo e quantidade de cada movimentação, data/hora, valor, motivo de saída, produto, fornecedor e endereço envolvido.

Como o sistema ainda está em fase inicial, o volume real de uso é baixo. Por isso, o plano de trabalho prevê duas fases: (1) uso de **dados simulados** (seed/massa de testes, já presente em `prisma/seed.ts`) para prototipar e validar as técnicas de mineração; (2) substituição progressiva por **dados reais de operação** assim que o sistema entrar em uso, mantendo a mesma estrutura de atributos.

Atributos relevantes por técnica

Grupo de dados	Atributos-chave	Uso analítico
Movimentações	tipo, quantidade, valor, data, produto, motivo de saída	Séries temporais, curva ABC, detecção de anomalias
Produtos	categoria, marca, estoque mín/máx, preço	Classificação ABC/XYZ, clustering por padrão de consumo
Lotes	data de fabricação/validade, status	Previsão de perdas por vencimento
Fornecedores	lead time implícito (data do pedido → entrada em estoque)	Otimização de ponto de pedido
Endereços de armazém	ocupação, corredor/posição	Associação produto–localização (slotting)

8.2 Planejamento Inicial das Técnicas



Figura 5 — Pipeline planejado de dados: da operação do sistema até os insights de IA.

Técnica	Objetivo no StockIQ	Status
Curva ABC	Classificar produtos por valor acumulado de movimentação, priorizando controle sobre os itens de maior impacto financeiro.	ENDPOINT PRONTO
Classificação XYZ	Classificar produtos pela variabilidade/regularidade da demanda, complementando a curva ABC (matriz ABC/XYZ).	PROTOTIPADO (MOCK)

Previsão de demanda (séries temporais)	Estimar a quantidade de saída futura por produto/categoria, antecipando risco de ruptura de estoque.	PROTOTIPADO (MOCK)
Regras de associação (Apriori/Market Basket)	Identificar produtos frequentemente movimentados juntos, apoiando decisões de slotting (posicionamento no armazém).	PLANEJADO
Clustering de produtos (k-means)	Agrupar produtos por padrão de consumo para sugerir automaticamente políticas de estoque mínimo/máximo.	PLANEJADO
Detecção de anomalias	Sinalizar movimentações de ajuste/perda fora do padrão cruzando com o log de auditoria.	PLANEJADO histórico

8.3 Ferramentas Cogitadas

- **Python** com `pandas` (preparação/limpeza), `scikit-learn` (clustering, classificação) e `statsmodels` / `Prophet` (séries temporais);
- **Jupyter Notebooks** para exploração e validação incremental dos modelos antes de qualquer automação;
- Extração via consultas diretas ao PostgreSQL nesta fase inicial; evolução futura para leitura do data lake em S3 (Capítulo 7) conforme o volume cresça;
- Possível uso futuro de serviços gerenciados de ML da AWS (ex.: SageMaker) caso o escopo de mineração se aprofunde além do que é viável rodar localmente.

8.4 Metodologia (CRISP-DM)

Etapa	Situação nesta sprint	
1. Entendimento do negócio	CONCLUÍDO	— objetivos de negócio definidos no Capítulo 2
2. Entendimento dos dados	CONCLUÍDO	— estrutura de dados definida no Capítulo 6
3. Preparação dos dados	PRÓXIMA SPRINT	— geração de massa simulada representativa
4. Modelagem	PRÓXIMA SPRINT	— primeiros protótipos de ABC/XYZ e forecast em notebook
5. Avaliação	SPRINTS SEGUINTEs	
6. Implantação	SPRINTS SEGUINTEs	— integração dos modelos ao módulo IA Analítica

STATUS DESTA ENTREGA

Base de dados e planejamento inicial das técnicas de mineração **concluídos**. O endpoint de curva ABC já implementado no back-end serve como primeira entrega concreta de análise de dados do projeto.

9. Status Consolidado da Sprint 1 e Próximos Passos

Entrega mínima da Sprint 1	Status	Observação
----------------------------	--------	------------

Definição de escopo e requisitos (funcionais e não funcionais)	CONCLUÍDO	16 RF + 10 RNF documentados (Cap. 2)
Modelagem inicial (casos de uso, arquitetura) 2	CONCLUÍDO	Diagrama de classes/sequência ficam para a Sprint
Estrutura inicial do back-end (framework + API configurada)	CONCLUÍDO	15 módulos / 67 endpoints, autenticado e testado
Protótipo inicial do front-end (telas estáticas)	CONCLUÍDO	20+ telas navegáveis; integração real com a API pendente
Banco de dados modelado (conceitual e lógico)	CONCLUÍDO	12 entidades, Prisma Migrate versionando o schema
Computação em Nuvem II — serviços e próximo passo	CONCLUÍDO	Arquitetura AWS definida; provisionamento é justificativa
Mineração de Dados — base e planejamento inicial	CONCLUÍDO	Base definida (dados do próprio sistema) + roteiro CRISP-DM

9.1 Próximos Passos (pós-Sprint 1)

1. Integrar o front-end à API real, substituindo os dados mockados por chamadas HTTP autenticadas;
2. Provisionar a infraestrutura AWS descrita no Capítulo 7 (VPC, EC2, Load Balancer, S3, primeira fila SQS);
3. Automatizar o deploy do back-end e do front-end nas instâncias EC2 (CI/CD simples ou scripts de deploy);
4. Gerar massa de dados simulada representativa para viabilizar os primeiros experimentos de mineração de dados;
5. Prototipar, em notebook Python, os primeiros modelos de classificação ABC/XYZ e previsão de demanda;
6. Elaborar diagrama de classes e diagramas de sequência dos fluxos críticos (movimentação, alerta, contagem de inventário);
7. Escrever testes automatizados para os módulos ainda não cobertos (fornecedores, clientes, lotes, inventário, relatórios).

10. Considerações Finais

A Sprint 1 cumpriu seu objetivo de estruturar a base técnica do StockIQ: escopo e requisitos documentados, modelagem inicial produzida, back-end funcional com autenticação e mais de 60 endpoints, protótipo navegável do front-end, banco de dados modelado e implantado via migrations, além das definições de infraestrutura em nuvem e do plano inicial de mineração de dados — ambos diretamente conectados ao domínio real do sistema (estoque, movimentações e lotes), e não tratados como exercícios teóricos isolados.

O maior risco identificado para as próximas sprints é a integração entre as três frentes — aplicação, nuvem e mineração de dados — dentro do prazo do semestre. Por isso, os próximos passos priorizam primeiro a integração front-end/back-end e o provisionamento básico da nuvem, para então liberar tempo da equipe para os experimentos de mineração de dados sobre uma base de dados real e em produção.